



UNIVERSIDAD NACIONAL DE INGENIERÍA

“Ciencia y Tecnología al Servicio del País”

RSA ALGORITHM



ES EL RESTO DE LA DIVISIÓN DE 2
NÚMEROS:

$$\begin{aligned} a &= c \times n + r \\ a \bmod(n) &= r \\ a &\equiv r \bmod(n) \end{aligned}$$

SUMA MODULAR:

$$\begin{aligned} 13 + 7 \bmod(5) &= 20 \bmod(5) = 0 \\ (13 \bmod(5) + 7 \bmod(5)) \bmod 5 &\rightarrow \\ &= 3 + 2 \bmod(5) = 0 \end{aligned}$$

MULTIPLICACIÓN MODULAR:

$$\begin{aligned} 9 \cdot 8 \bmod(7) &\rightarrow 72 = 10 \times 7 + 2 \\ 9 \cdot 8 \bmod(7) &= 2 \times 1 = 2 \\ 9 \cdot 8 &\equiv 2 \bmod(7) \end{aligned}$$

EXPONENCIACIÓN MODULAR:

$$\begin{aligned} 4^5 \bmod(7) &= ? \\ 4^1 \bmod(7) &\equiv 4 \bmod(7) \\ 4^2 \bmod(7) &\equiv (4)^2 \bmod(7) \equiv 2 \bmod(7) \\ 4^4 \bmod(7) &\equiv (2)^2 \bmod(7) \equiv 4 \bmod(7) \\ 4^5 &= 4^4 \cdot 4^1 \equiv 4 \cdot 4 \bmod(7) \equiv 2 \bmod(7) \end{aligned}$$

EL INVERSO MODULAR DE e MÓDULO m :
 $e \bmod(m)$ ES UN NÚMERO d (o e^{-1}) TAL
QUE AL MULTIPLICARLOS DAN

$$\begin{aligned} e \cdot d &\equiv 1 \bmod(m) \\ e \cdot e^{-1} &\equiv 1 \bmod(m) \end{aligned}$$



FUNCIÓN TONIENTE DE EULER:

$$\phi(n) = (p - 1) \times (q - 1)$$

$$\text{Si } n = p \times q = 5 \times 2 = 10$$

LOS NÚMEROS k COPRIMOS
MENORES QUE n , TAL QUE
 $\text{mcd}(k, n) = 1$

ESTOS SON: $k = \{1, 3, 7, 9\}$

CUANTOS NÚMEROS ENTEROS
POSITIVOS MENORES QUE n SON
COPRIMOS CON n , ES DECIR:

$$\phi(10) = (5 - 1) \times (2 - 1) = 4$$

INVERSOS MODULARES Y EL
TEOREMA DE EULER:



- LA LLAVE PÚBLICA e PUEDE SER UN NÚMERO ENTERO CORTO. LA LLAVE PRIVADA d DEBE TENER LA LONGITUD TOTAL DEL MÓDULO. POR LO TANTO, EL CIFRADO CON LA LLAVE PÚBLICA PUEDE SER SIGNIFICATIVAMENTE MÁS RÁPIDO QUE EL DESCIFRADO.
- EL PROGRESO EN LOS ALGORITMOS DE FACTORIZACIÓN Y EL HARDWARE UTILIZADO ES DIFÍCIL DE PREDECIR. POR TANTO, ES RECOMENDABLE USAR RSA CON UN MÓDULO DE 2048 BITS COMO MÍNIMO SI SE NECESITA UNA SEGURIDAD RAZONABLE A MEDIANO PLAZO, ESPECIALMENTE CON RESPECTO A ATACANTES EXTREMADAMENTE BIEN FINANCIADOS.
- RSA SE UTILIZA PRINCIPALMENTE EN:
 - EL TRANSPORTE DE LLAVES (ES DECIR, EL CIFRADO DE LLAVES) Y
 - LAS FIRMAS DIGITALES.
- CASO COMÚN DE EJEMPLO:
 - ALICE PUEDE ELEGIR UNA LLAVE SIMÉTRICA (LLAVE DE SESIÓN) PARA CIFRAR UN DATO.
 - ALICE ENTONCES CIFRA LA LLAVE DE SESIÓN UTILIZANDO LA LLAVE PÚBLICA DE BOB.
 - LUEGO, BOB DESCIFRA EL MENSAJE CON SU LLAVE PRIVADA, Y OBTIENE LA LLAVE DE SESIÓN.
 - BOB ENTONCES PUEDE USAR LA LLAVE DE SESIÓN PARA DESCIFRAR EL DATO ENVIADO.

7

**ALGORITMO DESCRIPCIÓN**

GENERACIÓN DE PAR DE LLAVES RSA

Salida: llave pública y llave privada

- + Elegir dos números primos de gran longitud y
- + Calcular
- + Calcular
- + Seleccionar el exponente público de tal forma que
- + Calcular la llave privada de tal forma que

CIFRADO RSA

Dada la llave pública y el mensaje plano

La función de cifrado es:

Donde

DESCIFRADO RSA

Dada la llave privada y el mensaje cifrado

La función de descifrado es:

Donde

8



ETAPA	OPERACIÓN	CÁLCULO / MÉTODO	RESULTADO
Primos		elegidos	
Módulo			
Función			
Exponente público			
Exponente privado		Inverso de 3 mod 40 (Euclides extendido)	
Llave pública			
Llave privada			
Mensaje		elegido $< n$	
Cifrado			
Descifrado		Exponenciación modular:	



ALGORITMO	DESCRIPCIÓN
GENERACIÓN DE PAR DE LLAVES RSA	Salida: llave pública y llave privada + Elegir dos números primos de gran longitud y + Calcular + Calcular + Selecciona el exponente público de tal forma que + Calcular la llave privada de tal forma que
FIRMA RSA	Dada la llave privada y el mensaje plano La función de firma es: Donde
VERIFICACIÓN RSA	Dada la llave pública y el mensaje firmado La función de verificación es: Donde



Basic RSA Digital Signature Protocol

Alice

Bob

$$k_{pr} = d, k_{pub} = (n, e)$$

(n, e)

compute signature:

$$s = \text{sig}_{k_{pr}}(x) \equiv x^d \pmod{n}$$

(x, s)

verify: $\text{ver}_{k_{pub}}(x, s)$

$$x' \equiv s^e \pmod{n}$$

$$x' \begin{cases} \equiv x \pmod{n} & \Rightarrow \text{valid signature} \\ \not\equiv x \pmod{n} & \Rightarrow \text{invalid signature} \end{cases}$$

11



12